

存储的核心作用在于持久化存储大量的数据。

数据量越来越大，给数据库带来很大的存储负载、IO 负载、计算负载，因此使用负载均衡降低数据库的负载。

mysql 分库分表

mysql 的实例、表作为单个存储单元，负载均衡需要上层实现。

常用的 mysql 负载均衡策略为分库分表。

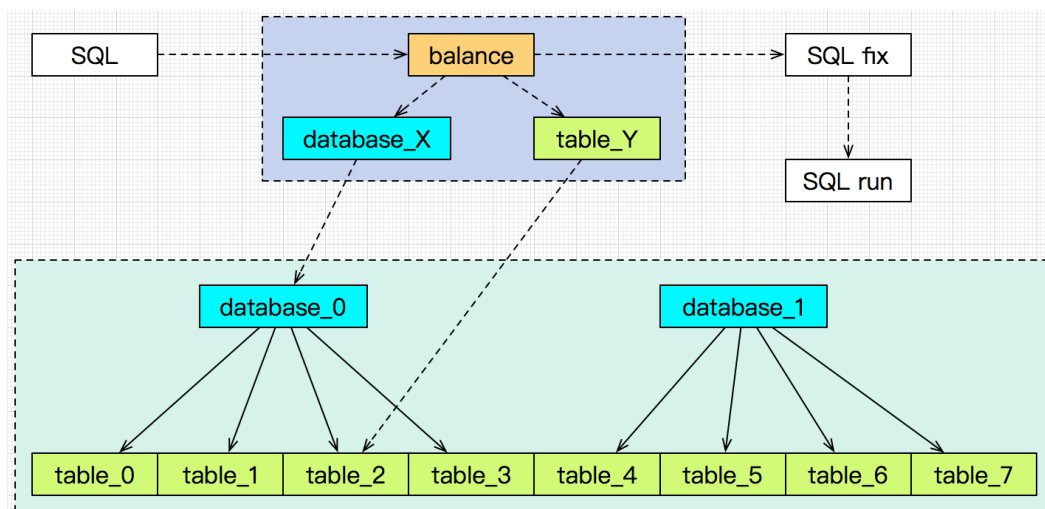
分库规则：

- 功能：把某个业务的数据，存储到多个 mysql 实例。
- 一般按照业务拆分，比如订单表有 8 个 mysql 实例，商品表有 16 个 mysql 实例。
- 为了容错、扩展，可以把每个 mysql 实例分别放在一个机器，实现物理隔离。
- 业务的数据量达到某个量级，才需要分库。分库提高配置、管理的复杂度。

分表规则：

- 哈希法，使用关键字段，哈希取模作为表的后缀。比如，order_0, order_1, order_2, order_3 。
- 哈希法，常用于核心表，支持高效的读写操作，使用核心字段拆分。比如，商品表按照 seller_id 拆分，订单表按照 buyer_id 拆分。
- 日期法，使用年月日作为表的后缀。
- 日期法，常用于次要表、日志表，一般不支持修改，不支持跨表查询。比如用户操作购物车的日志表，user_cart_log_2021、user_cart_log_2022、user_cart_log_2023 。

分库分表的架构图：



写代码，模拟分库分表功能：

- 分库分表的规则。这里做了简化，包含库的地址、表的范围。

```
public class Database {
    String databaseAddr;// 实例的地址
    int tableIndexBegin;// 表下标, 开始
    int tableIndexEnd;// 表下标, 结束
}
```

- 配置数据源。多个分库，多个分表。

```
static String tableName = "product";
```

```

static int databaseCount = 2;// 分库
static int tableCount = 8;// 分表
static List<Database> databaseList = new ArrayList<>();

// 配置数据源
public static void configDatasource() {
    // 分库有 2 个。每个分库有 4 个分表
    Database database0 = new Database().setDatabaseAddr("192.168.3.100:3306")
        .setTableIndexBegin(0).setTableIndexEnd(3);
    Database database1 = new Database().setDatabaseAddr("192.168.3.101:3306")
        .setTableIndexBegin(4).setTableIndexEnd(7);
    databaseList.add(database0);
    databaseList.add(database1);
}

```

- 分库分表的核心逻辑。使用字段 sellerId 找到对应的分库、分表，然后执行 sql。

```

// 分库分表。依赖字段 sellerId
public static void balanceSql(long sellerId, String sql) {
    // 下标。
    int tableIndex = (int) (sellerId % tableCount);
    int databaseIndex = tableIndex / (tableCount / databaseCount) % databaseCount;
    // 库。
    Database database = databaseList.get(databaseIndex);
    // 表。真实的表名
    String tableNameFix = tableName + "_" + tableIndex;
    // 替换表名、字段。这里仅为示例。
    String sqlFix = sql.replaceAll(tableName, tableNameFix);
    sqlFix = sqlFix.replaceAll("#0", sellerId + "");
    System.out.println(sellerId + "    tableIndex= " + tableIndex
        + "    database= " + database.databaseAddr + " \n        SQL= " + sqlFix);
}

```

- 运行。先配置数据源，后运行 sql。这里分别测试了查询语句、修改语句。

```

public static void main(String[] args) {
    configDatasource();
    // 查询语句
    balanceSql(3001, "select * from product where seller_id = #0 and name like '%orange%'");
    balanceSql(5638, "select * from product where seller_id = #0 and price > 100");
    // 修改语句
    balanceSql(6091, "update product set price = 200 where seller_id = #0 and id = 70007");
    balanceSql(9765, "delete from product where seller_id = #0 and id = 80008");
}

```

- 输出。根据不同的字段 sellerId，找到分库、分表，sql 中的表名被替换了。

```

3001    tableIndex= 1    database= 192.168.3.100:3306
        SQL= select * from product_1 where seller_id = 3001 and name like '%orange%'

```

```

5638  tableIndex= 6  database= 192.168.3.101:3306
      SQL= select * from product_6 where seller_id = 5638 and price > 100
6091  tableIndex= 3  database= 192.168.3.100:3306
      SQL= update product_3 set price = 200 where seller_id = 6091 and id = 70007
9765  tableIndex= 5  database= 192.168.3.101:3306
      SQL= delete from product_5 where seller_id = 9765 and id = 80008

```

思考：存储 5000 万行数据，1 个表存 5000 万行，2 个表各存 2500 万行，这 2 种方式有什么优点缺点？
 思考：分库数量、分表数量，为什么使用 2 的 N 次方？

hbase 分库分表

hbase 作为分布式数据库，自动实现负载均衡，不需要手动分库分表。

hbase 负载均衡使用行键分片、动态平衡。

● 行键分片：

行键 rowkey 是 hbase 表的主键。

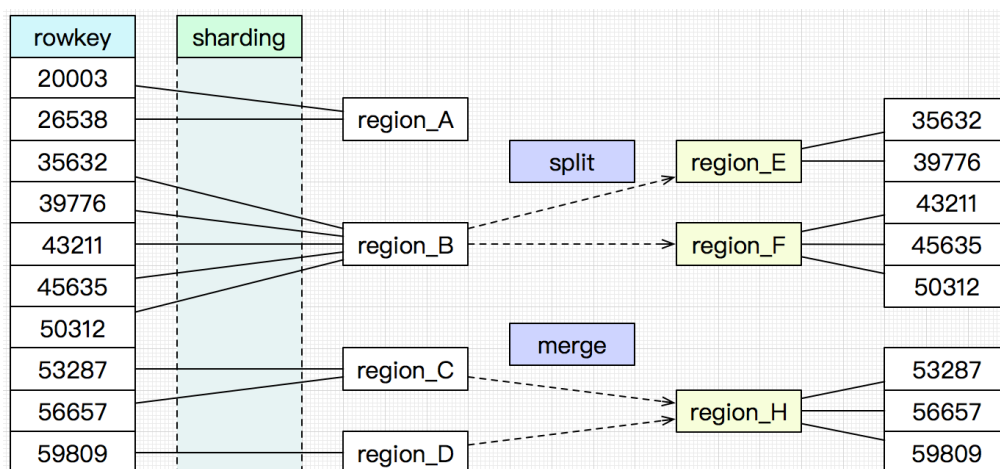
表的数据，按照 rowkey 切分为多个分片区间，每个分片区间存储在一个区域 region。

● 动态平衡：

如果某个 region 的数据过多，则自动拆分为多个 region。

如果多个连续 region 的数据过少，则自动合并为 1 个 region。

hbase 负载均衡的架构图：



如上图。

rowkey 包含很多值，rowkey 有序。

rowkey 分片。region 有多个，每个 region 对应 N 个 rowkey。

region_B 对应 5 个 rowkey，负载过高，拆分为 region_F。

region_C、region_D 分别对应 2/1 个 rowkey，负载过低，合并为 region_H。